

Software: This is a set program.

Operating System: OS is a system software.

Program: This is a set of instructions which is in any computer language.

System software: System software is a set of programs that controls a computer's hardware and application programs. It acts as an interface between the hardware and the user applications, allowing them to communicate with each other.

Type of OS.

1.Single User Single-Tasking: Designed for one user to perform a single task at a time. For ex early version of MS-DOS.

2.Single User Multi-Tasking: Allow a single user to perform multiple tasks simultaneously. For ex windows, macOS.

3.Mutli-user: Supports multiple users accessing the system at the same time. For ex Linux, UNIX.

4.Realtime OS: These are designed for systems that require immediate processing and responses. For ex VxWorks, QNX.

5.Mobile OS: This is a specialized OS for mobile devices, offering touch interfaces and energy efficient operations. For ex android, IOS.

Components of OS.

1.Kernel: The Linux kernel is a core component of a Linux OS and is the core interface between a computer's hardware and its processes.

2.Device drivers: This are software components that enables the OS to communicate with hardware devices. They act as an interface between hardware and kernel.

3.System libraries: This are precompiled code that provides the essential functions and services to applications and the kernel.

4.User interface: This refers to the means by which users interact with the system, typically through a CLI and GUI. It allows user to execute commands and run programs and manage system resources efficiently.

Most popular operating system: Windows, Linux, macOS, Android, IOS, Unix.

Computing System: A Linux system is a computing system that uses the Linux OS to manage hardware resources, launch applications and provide a user interface.

Types of computing system:

1.Dekstop Environment.

2.Server Environment.

3.Cloud Computing Environment.

4.Mobile Computing Environment.

5.Embedded Computing Environment.

6.Distributed Computing Environment.

Difference between CLI and GUI:

CLI	GUI
CLI is difficult to use.	GUI is easy to use.
Consumes low memory.	Consumes more memory.
In CLI we can obtain high precision.	In GUI we can obtain low precision.
CLI OS needs only keyboard.	GUI OS requires keyboard and mouse.
Input is entered only at a command prompt.	Input can be entered anywhere on the screen.
CLI doesn't have graphics.	GUI does have graphics.
CLI's appearance cannot be changed/modified.	GUI's appearance can be changed/modified.
It is faster than GUI.	It is slower than CLI.

Key functions of OS.

1.Memory management: In computer system, we have primary memory, which is also called as main memory and it is made by the sequence of bytes. Example, it allocates the main memory for program execution and when program is terminated/completed, then it deallocates the memory.

2.Device management: The OS manages the communication between the system and peripheral devices connected to I/O port of system. Example, it controls the I/O device like printer, USB ports, microphone, keyboard, etc.

3.File management: Keeps track of where data is stored, user access settings and state of each file. A file system is organized into directories for efficient or easy navigation and usage. These directories may contain other directories and other files. An OS keeps track of information regarding the creation, deletion, transfer, copy, and storage of files in an organized way.

4.Security and Privacy: The operating system uses password protection to protect user data and similar other techniques. It also prevents unauthorized access to programs and user data. The operating system provides various techniques which assure the integrity and confidentiality of user data.

5.Process management: In a multi-programming environment, the OS decides the order in which processes have access to the processor, and how much processing time each process has. This function of OS is called Process scheduling.

6.Networking: This is an OS and is the process of connecting devices and systems to share resource and exchange data.

7.Interrupt handling: This is an OS that manages interrupts and their handlers to respond to unexpected events that disrupt the normal flow of instruction program execution.

OS Operations.

1.Process management: The OS manages processes, which are instances of program in execution. It has four important aspects i.e process creation, process scheduling (FCFS), process termination, process synchronization. It schedules processes, allocate resources and ensure smooth execution without conflict.

2.Memory management: The OS allocates and manages RAM to running processes, ensuring sufficient use of memory and preventing conflicts like memory leaks. It has four important aspects i.e memory allocation, memory deallocation, memory protection, memory virtual management.

3.File management: It controls the reading, writing and storage of data on disk drivers, providing file systems for organizing files and directories. It has five important aspects i.e file creation, file deletion, file reading and writing, file navigation, file i/p and o/p.

4.Security and Access control: It ensures that the system and its data are protected from unauthorized access through authentication, encryptions and user permissions.

5.Input and Output management: The OS manages the i/p and o/p devices to facilitate user interactions and data exchange. It also schedules i/p and o/p enter by the user.

OS Services.

1.Program Execution: It is the OS that manages how a program is going to be executed. It loads the program into the memory after which it is executed. The order is depend on the CPU scheduling algorithm.

2.I/O Operations: OS manages the i/p-o/p operations and establishes communication between the user and device drivers. Device drivers (software) managed by the OS. It also provides access to i/p-o/p devices to program when needed.

3.File Management: The OS help in managing files. If a program needs access to a file, it is the OS that grants access. These permissions include read-only, read-write, etc. It also provides a platform for the user to create and delete files. The OS decides how the data should be manipulated and stored.

4.Communication between processes: The OS manages the communication between processes. Communication between processes include data transfer among them. If the processes are not on the same computer but connected through a computer network, then also their communication is managed by the OS itself.

5.Error handling: The OS handles the error occurring in the CPU, in the i/p-o/p devices, etc. It also ensures that an error does not occur frequently and fixes the error. It also looks for any type of error/bugs that can occur while doing any task.

6.Networking: These services enable communication between devices on a network, such as connecting to the internet, sending and receiving data packets and managing network connection.

7.User interface: User interface is essential and all OS provides it. Users either interact with the OS through CLI and GUI.

OS Structure: The structure of an OS refers to how the components of the OS are organized and interact with each other. The structure of an OS can affect it ease of use, efficiency and how easily it can be modified.

System Centric Services:

- 1.Resource Allocation.
- 2.Security and Protection.
- 3.System Monitoring.
- 4.System services.

User in Linux.

1.Root User: This is also known as superuser. It does have any restrictions. It has the highest level of access and privileges, including the ability to modify system files and install software.

2.Regular User: This user are created for everyday task. Have limited access and has home directory. They can only perform certain actions within their designated scope.

3.System User: They are non-human user. A system user in Linux is an internal user created during the installation of the operating system or software applications. System users are used to run system services and tasks, and typically do not have a login shell.

Common command for user management.

- 1.Adding user: adduser/useradd.
- 2.Password storage to change and set users password: /etc/Passwd.
- 3.User directory define in: /etc/shadow where shadow is directory.
- 4.Delete user: deluser/userdel.
- 5.Modify user detail: usermod.
- 6.To check login user: who/w.
- 7.To display users id: id.

Interfaces in Linux.

1.GUI: A visual representation that uses icons, buttons, and windows to interact with the system. GUIs are more intuitive and user-friendly, making them suitable for novice users. However, they consume more space and resources than CLI. Ex GNOME, KTE PLASMA.

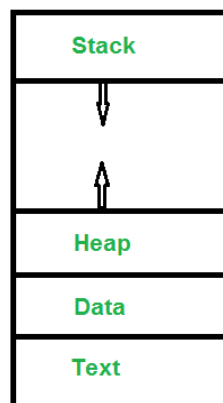
2.CLI: A text-based representation that uses a terminal to type commands to interact with the system. CLIs provide more control over the system and are well-suited for older hardware or systems with limited processing power. However, they usually require command and syntax expertise.

Process in memory:

A program is a passive entity (a file with instructions stored on disk). A process is an active entity with a program counter and associated resources. A program becomes a process when it is loaded into memory. Two common ways to load an executable file:

- i. Double-clicking the file's icon.
- ii. Entering its name on the command line.

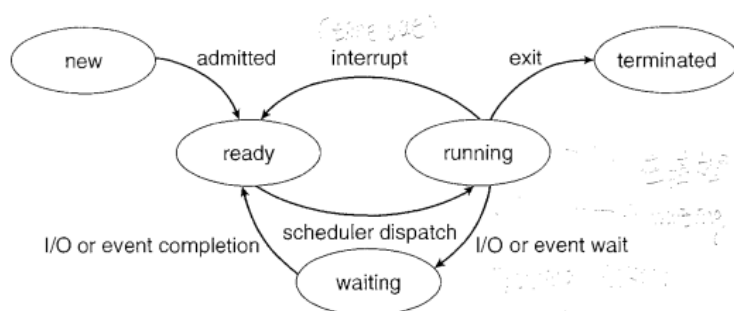
A process in memory is divided into several distinct sections, each serving a different purpose. Here's how a process typically looks in memory:



1. Text Section: A text or code segment contains executable instructions. It is typically a read only section
2. Stack: The stack contains temporary data, such as function parameters, returns addresses, and local variables.
3. Data Section: Contains the global variable.
4. Heap Section: Dynamically memory allocated to process during its run time.

Five state process model:

The five-state process lifecycle is an expanded version of the two-state model. The two-state model works well when all processes in the not running state are ready to run. However, in some operating systems, a process may not be able to run because it is waiting for something, like input or data from an external device. To handle this situation better, the not running state is divided into two separate states:



New: This state represents a newly created process that hasn't started running yet. It has not been loaded into the main memory, but its process control block (PCB) has been created, which holds important information about the process.

Ready: A process in this state is ready to run as soon as the CPU becomes available. It is waiting for the operating system to give it a chance to execute.

Running: This state means the process is currently being executed by the CPU. Since we're assuming there is only one CPU, at any time, only one process can be in this state.

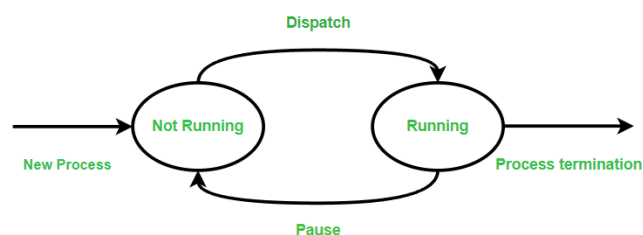
Blocked/Waiting: This state means the process cannot continue executing right now. It is waiting for some event to happen, like the completion of an input/output operation (for example, reading data from a disk).

Exit/Terminate: A process in this state has finished its execution or has been stopped by the user for some reason. At this point, it is released by the operating system and removed from memory.

Two state process model:

The simplest way to think about a process's lifecycle is with just two states:

- i. Running: This means the process is actively using the CPU to do its work.
- ii. Not Running: This means the process is not currently using the CPU. It could be waiting for something, like user input or data, or it might just be paused.



When a new process is created, it starts in the not running state. Initially, this process is kept in a program called the dispatcher.

Steps:

1. Not Running State: When the process is first created, it is not using the CPU.
2. Dispatcher Role: The dispatcher checks if the CPU is free (available for use).
3. Moving to Running State: If the CPU is free, the dispatcher lets the process use the CPU, and it moves into the running state.
4. CPU Scheduler Role: When the CPU is available, the CPU scheduler decides which process gets to run next. It picks the process based on a set of rules called the scheduling scheme, which varies from one operating system to another.